

GUÍA DE LABORATORIO N° 8

Autenticación con Laravel Breeze

y Personalización del Starter Kit

INF560 · Desarrollo Web Backend*Material de Laboratorio — Construcción de Aplicación Web con Laravel 13***CONTENIDO DE LA GUÍA**

1. Instalación de Laravel Breeze (Blade) en un proyecto Laravel 13
2. El scaffolding de autenticación: rutas, controladores y vistas publicadas
3. Personalización 1 — Branding: logo, colores Tailwind y layout
4. Personalización 2 — Localización completa de la interfaz al español
5. Personalización 3 — Campo de rol en el registro de usuarios
6. Personalización 4 — Navbar por rol, middleware y redirección por rol

COMPETENCIA DE LA GUÍA

Al finalizar la guía, el estudiante será capaz de incorporar un sistema de autenticación funcional a una aplicación Laravel mediante el starter kit Laravel Breeze, y de personalizarlo a nivel de identidad visual, idioma, modelo de datos y control de acceso, comprendiendo que Breeze publica todo su código en el proyecto y que ello otorga control total sobre rutas, controladores y vistas de autenticación.

Docente: M. Sc. Huáscar Fedor Gonzales Guzmán · Universidad Autónoma Tomás Frías (UATF) — Potosí, Bolivia

1. Objetivos de la Guía

Objetivo general

Construir una aplicación web en Laravel 13 con autenticación basada en Laravel Breeze y aplicar un conjunto de personalizaciones que adapten el starter kit a las necesidades de marca, idioma y control de acceso del proyecto.

Objetivos específicos

- Instalar Laravel Breeze con la pila Blade sobre un proyecto Laravel 13 recién creado.
- Identificar y describir los archivos que Breeze publica en el proyecto (rutas, controladores, vistas y componentes).
- Personalizar la identidad visual de las vistas de autenticación (logo y colores con Tailwind CSS).
- Localizar al español la interfaz de Breeze y los mensajes de validación del framework.
- Ampliar el modelo de usuario con un atributo de rol y reflejarlo en el formulario de registro de forma segura.
- Implementar un menú de navegación y una redirección posteriores al inicio de sesión condicionados por el rol del usuario.

2. Prerrequisitos

Entorno de desarrollo. Antes de comenzar, verifica que dispones del siguiente entorno:

Herramienta	Versión recomendada	Verificación
PHP	8.3 o superior	php -v
Composer	2.x	composer --version
Node.js / npm	20 LTS o superior	node -v
PostgreSQL	14 o superior	psql --version
Laravel Installer	Última	laravel --version

Conocimientos previos. Esta guía asume el manejo de rutas, controladores, vistas Blade, migraciones y Eloquent trabajados en las guías anteriores del curso. No requiere haber completado el proyecto library-app: se construye un proyecto nuevo e independiente.

Proyecto de esta guía

Se desarrolla una aplicación nueva llamada portal-breeze: un portal académico mínimo con tres roles (admin, docente y estudiante) donde cada rol, tras autenticarse, es dirigido a su propio panel.

3. Laravel Breeze: qué es y su estado actual

Laravel Breeze es una implementación mínima y sencilla de todas las funciones de autenticación de Laravel: inicio de sesión, registro, restablecimiento de contraseña, verificación de correo y confirmación de contraseña, además de una página de perfil. Su capa de vistas por defecto se compone de plantillas Blade estilizadas con Tailwind CSS.

Característica clave: publica su propio código

A diferencia de un paquete tradicional que se ejecuta desde la carpeta vendor, Breeze copia (publica) sus rutas, controladores, vistas y componentes dentro de tu proyecto. Por eso puedes editarlo libremente: todo lo que personalizaremos en esta guía son archivos que viven en tu repositorio.

Un detalle importante de contexto

Desde Laravel 12, Breeze y Jetstream dejaron de aparecer en el instalador oficial (laravel new ahora ofrece kits para React, Vue y Livewire) y dejaron de anunciarse en la documentación principal. Sin embargo, Breeze sigue mantenido y publicado: su versión actual en Packagist declara compatibilidad con Laravel 11, 12 y 13. Para nuestro enfoque monolítico con Blade, sigue siendo la opción más directa; simplemente se instala de forma manual con Composer.

Implicación práctica

Cuando crees el proyecto con laravel new, elige la opción None (sin starter kit). Breeze NO aparecerá en esa lista. Lo instalaremos después con composer require laravel/breeze --dev, como se indica en el Paso 2.

4. Desarrollo paso a paso

► Paso 1 — Crear el proyecto Laravel 13 y configurar PostgreSQL

Crea el proyecto con el instalador de Laravel. Cuando el asistente lo pregunte, selecciona None como starter kit, PostgreSQL como base de datos y el framework de pruebas que prefieras.

```
TERMINAL
# Crear el proyecto (elige: None, PostgreSQL)
laravel new portal-breeze

cd portal-breeze
```

Crea la base de datos en PostgreSQL y ajusta el archivo .env:

```
TERMINAL
# Crear la base de datos (psql o tu cliente preferido)
createdb portal_breeze

.ENV
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=portal_breeze
DB_USERNAME=postgres
DB_PASSWORD=tu_password
```

Comprueba la conexión

Ejecuta `php artisan migrate` al final del Paso 2 (no ahora). Si la conexión falla, revisa usuario/contraseña y que el servicio de PostgreSQL esté activo.

► Paso 2 — Instalar Laravel Breeze (pila Blade)

Instala el paquete como dependencia de desarrollo y ejecuta el comando de scaffolding. El argumento `blade` selecciona directamente la pila de plantillas Blade (también puedes ejecutar el comando sin argumentos para elegir de forma interactiva).

```
TERMINAL
composer require laravel/breeze --dev

php artisan breeze:install blade
```

A continuación, ejecuta las migraciones y compila los assets:

```
TERMINAL
php artisan migrate
npm install
npm run dev
```

¿Qué se publicó? Estos son los archivos que tendrás que conocer (y que personalizaremos):

Ubicación	Contenido
routes/auth.php	Rutas de login, registro, recuperación, verificación.
app/Http/Controllers/Auth/	Controladores de cada flujo de autenticación.
resources/views/auth/	Vistas: login, register, forgot-password, reset-password...
resources/views/layouts/	Layouts app.blade.php (autenticado) y guest.blade.php.
resources/views/components/	Componentes: application-logo, primary-button, nav-link...
resources/views/dashboard.blade.php	Panel mostrado tras iniciar sesión.

► Paso 3 — Probar el scaffolding por defecto

Levanta el servidor de desarrollo (mantén npm run dev corriendo en otra terminal):

```
TERMINAL  
php artisan serve
```

Abre <http://127.0.0.1:8000>. Verás enlaces de Log in y Register en la página de bienvenida. Regístrate con un usuario de prueba: tras el registro, Breeze te llevará al dashboard con el mensaje «You're logged in!». Con esto confirmamos que la autenticación funciona; ahora comienza la personalización.

► Paso 4 — Personalización 1: Branding (logo y colores)

4.1 Reemplazar el logo

Edita el componente `resources/views/components/application-logo.blade.php` y sustituye el SVG por uno propio. Como las clases CSS se aplican desde fuera vía `$attributes`, mantén el SVG simple:

```
RESOURCES/VIEWS/COMPONENTS/APPLICATION-LOGO.BLADE.PHP
<svg {{ $attributes }} viewBox="0 0 48 48" fill="none"
  xmlns="http://www.w3.org/2000/svg">
  <rect x="6" y="8" width="36" height="32" rx="4"
    fill="currentColor" opacity="0.12"/>
  <path d="M10 12h12a4 4 0 0 1 4 4v20a4 4 0 0 0-4-4H10V12z"
    fill="currentColor"/>
  <path d="M38 12H26a4 4 0 0 0-4 4v20a4 4 0 0 1 4-4h12V12z"
    fill="currentColor" opacity="0.55"/>
</svg>
```

Sobre el color del logo

El SVG usa `currentColor`, por lo que toma el color del texto. En la vista, controla su color con una clase como `text-brand` (definida abajo) o `fill-current`.

4.2 Definir los colores de marca

La forma de definir colores personalizados depende de la versión de Tailwind que Breeze haya instalado. Verifícala primero:

```
TERMINAL
npm ls tailwindcss
# o revisa la versión en package.json
```

Opción A — Tailwind v3 (existe el archivo `tailwind.config.js`). Añade la paleta dentro de `theme.extend.colors`:

```
TAILWIND.CONFIG.JS
// tailwind.config.js
export default {
  content: [
    './resources/**/*.blade.php',
    './resources/**/*.js',
  ],
  theme: {
    extend: {
      colors: {
        brand: {
          DEFAULT: '#1E3A5F',
          light: '#2D5F8A',
          dark: '#142C49',
          accent: '#0F766E',
        },
      },
    },
  },
  plugins: [],
};
```

Opción B — Tailwind v4 (no hay `tailwind.config.js`; la configuración vive en el CSS). Declara los colores como variables de tema en `resources/css/app.css`:

```
RESOURCES/CSS/APP.CSS
/* resources/css/app.css */
@import "tailwindcss";
@source "../**/*.blade.php";

@theme {
  --color-brand:          #1E3A5F;
  --color-brand-light:    #2D5F8A;
  --color-brand-dark:     #142C49;
  --color-brand-accent:   #0F766E;
}
```

Ambas opciones generan las mismas clases

Tras configurar la marca, en cualquiera de las dos versiones podrás usar utilidades como `bg-brand`, `text-brand`, `hover:bg-brand-dark`, etc.

4.3 Aplicar la marca al botón y al fondo de autenticación

Cambia el color del botón principal una sola vez editando `resources/views/components/primary-button.blade.php`:

```
RESOURCES/VIEWS/COMPONENTS/PRIMARY-BUTTON.BLADE.PHP
<button {{ $attributes->merge([
  'type' => 'submit',
  'class' => 'inline-flex items-center px-4 py-2 bg-brand
              border border-transparent rounded-md font-semibold
              text-xs text-white uppercase tracking-widest
              hover:bg-brand-dark focus:bg-brand-dark
              transition ease-in-out duration-150',
]) }}>
  {{ $slot }}
</button>
```

Y aplica el fondo de marca a la zona de invitado en `resources/views/layouts/guest.blade.php` reemplazando la clase de fondo del contenedor (p. ej. `bg-gray-100`) por una con tu marca:

```
RESOURCES/VIEWS/LAYOUTS/GUEST.BLADE.PHP
<div class="min-h-screen flex flex-col items-center pt-6
           sm:justify-center sm:pt-0
           bg-gradient-to-b from-brand to-brand-dark">
  ...
</div>
```

Recompila para ver los cambios

Mantén `npm run dev` en ejecución; al guardar los archivos, Vite recompila y refresca el navegador automáticamente.

► Paso 5 — Personalización 2: Localización al español

5.1 Fijar el idioma de la aplicación

En Laravel 11+ el idioma se controla desde `.env`. Ajusta estas claves:

```
.ENV
APP_LOCALE=es
APP_FALLBACK_LOCALE=en
APP_FAKER_LOCALE=es_ES
```

5.2 Publicar y traducir los archivos de idioma del framework

La carpeta `lang/` no viene incluida por defecto. Publícala y crea la versión en español:

```
TERMINAL
php artisan lang:publish    # crea lang/en/*.php
mkdir -p lang/es
```

Crea `lang/es/auth.php` y `lang/es/passwords.php` con las traducciones de los mensajes de autenticación:

```
LANG/ES/AUTH.PHP
<?php // lang/es/auth.php
return [
    'failed' => 'Estas credenciales no coinciden con nuestros registros.',
    'password' => 'La contraseña proporcionada es incorrecta.',
    'throttle' => 'Demasiados intentos. Intente nuevamente en :seconds segundos.',
];
```

Para los cientos de mensajes de `validation.php`, lo recomendable es usar el paquete de traducciones de la comunidad en lugar de escribirlos a mano:

```
TERMINAL
composer require laravel-lang/common --dev
php artisan lang:add es
php artisan lang:update
```

5.3 Traducir los textos de las vistas de Breeze

Las vistas de Breeze envuelven muchos textos con la función `__()`. Crea un archivo de traducciones JSON `lang/es.json` que mapee cada cadena en inglés a su equivalente en español:

```
LANG/ES.JSON
{
    "Name": "Nombre",
    "Email": "Correo electrónico",
    "Password": "Contraseña",
    "Confirm Password": "Confirmar contraseña",
    "Remember me": "Recordarme",
    "Log in": "Iniciar sesión",
    "Register": "Registrarse",
    "Already registered?": "¿Ya tienes una cuenta?",
    "Forgot your password?": "¿Olvidaste tu contraseña?",
    "Log Out": "Cerrar sesión",
    "Dashboard": "Panel",
    "Profile": "Perfil"
}
```


Textos que no usan `__()`

Algunas cadenas están escritas directamente en las plantillas (por ejemplo «You're logged in!» en `dashboard.blade.php`). Edítalas directamente en la vista o envuélvelas con `{{ __('...') }}` y añade la entrada al JSON.

► Paso 6 — Personalización 3: Campo de rol en el registro

6.1 Migración: añadir la columna role

TERMINAL

```
php artisan make:migration add_role_to_users_table
```

En la migración generada, dentro de `up()`:

```
DATABASE/MIGRATIONS/..._ADD_ROLE_TO_USERS_TABLE.PHP
Schema::table('users', function (Blueprint $table) {
    $table->string('role')->default('estudiante')->after('email');
});

// down():
Schema::table('users', function (Blueprint $table) {
    $table->dropColumn('role');
});
```

Por qué string y no enum de base de datos

Una columna string con un valor por defecto es portable y simple de migrar en PostgreSQL. La restricción de valores válidos la haremos en la validación y el modelo, no en el motor de base de datos.

Ejecuta la migración:

TERMINAL

```
php artisan migrate
```

6.2 Modelo User: fillable y helpers de rol

En `app/Models/User.php`, agrega `role` a `$fillable` y define constantes y métodos de ayuda:

```
APP/MODELS/USER.PHP
protected $fillable = ['name', 'email', 'password', 'role'];

public const ROLES = ['admin', 'docente', 'estudiante'];

// Roles que un usuario puede elegir al registrarse
public const SELF_ROLES = ['docente', 'estudiante'];

public function isAdmin(): bool
{
    return $this->role === 'admin';
}

public function hasRole(string ...$roles): bool
{
    return in_array($this->role, $roles, true);
}
```

6.3 Vista de registro: selector de rol

En `resources/views/auth/register.blade.php`, añade un `<select>` después del campo de correo:

RESOURCES/VIEWS/AUTH/REGISTER.BLADE.PHP

```

<!-- Rol -->
<div class="mt-4">
    <x-input-label for="role" :value="__('Tipo de usuario')"/>
    <select id="role" name="role"
        class="block mt-1 w-full border-gray-300 rounded-md shadow-sm">
        @foreach (\App\Models\User::SELF_ROLES as $r)
            <option value="{{ $r }}" @selected(old('role') === $r)>
                {{ ucfirst($r) }}
            </option>
        @endforeach
    </select>
    <x-input-error :messages="$errors->get('role')"/>
</div>

```

6.4 Controlador de registro: validar y guardar el rol

En `app/Http/Controllers/Auth/RegisteredUserController.php`, actualiza la validación y la creación del usuario:

```

APP/HTTP/CONTROLLERS/AUTH/REGISTEREDUSERCONTROLLER.PHP
use Illuminate\Validation\Rule;

$request->validate([
    'name' => ['required', 'string', 'max:255'],
    'email' => ['required', 'string', 'lowercase', 'email',
        'max:255', 'unique:'.User::class],
    'password' => ['required', 'confirmed', Rules\Password::defaults()],
    'role' => ['required', Rule::in(User::SELF_ROLES)],
]);

$user = User::create([
    'name' => $request->name,
    'email' => $request->email,
    'password' => Hash::make($request->password),
    'role' => $request->role,
]);

```

Seguridad: nunca confíes en el rol enviado por el cliente

La regla `Rule::in(User::SELF_ROLES)` impide que un usuario malicioso se autoasigne el rol `admin` manipulando el formulario. El rol administrador se crea únicamente desde un seeder controlado, como se muestra a continuación. Asignar privilegios a partir de datos no validados del cliente es una vulnerabilidad de control de acceso.

6.5 Seeder del administrador

Crea el usuario administrador desde `database/seeder/DatabaseSeeder.php`:

```

DATABASE/SEEDERS/DATABASESEEDER.PHP
use App\Models\User;
use Illuminate\Support\Facades\Hash;

public function run(): void
{
    User::updateOrCreate(
        ['email' => 'admin@uatf.edu.bo'],
        [
            'name' => 'Administrador',
            'password' => Hash::make('password'),
            'role' => 'admin',
        ]
    );
}

```

```
    ]  
    );  
}
```

TERMINAL

```
php artisan db:seed
```

► Paso 7 — Personalización 4: Navbar, middleware y redirección por rol

7.1 Middleware de verificación de rol

TERMINAL

```
php artisan make:middleware EnsureUserHasRole
```

En `app/Http/Middleware/EnsureUserHasRole.php`:

APP/HTTP/MIDDLEWARE/ENSUREUSERHASROLE.PHP

```
public function handle(Request $request, Closure $next, string ...$roles): Response
{
    if (! $request->user() || ! $request->user()->hasRole(...$roles)) {
        abort(403, 'No tienes permiso para acceder a esta sección.');
```

```
        return $next($request);
    }
}
```

Registra el alias `role` en `bootstrap/app.php`:

BOOTSTRAP/APP.PHP

```
->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'role' => \App\Http\Middleware\EnsureUserHasRole::class,
    ]);
});
```

7.2 Controlador que redirige según el rol

TERMINAL

```
php artisan make:controller DashboardController
```

Breeze redirige al usuario a la ruta `dashboard` tras el login. Haremos que esa ruta despache hacia el panel correspondiente al rol:

APP/HTTP/CONTROLLERS/DASHBOARDCONTROLLER.PHP

```
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;

class DashboardController extends Controller
{
    public function index(Request $request): RedirectResponse
    {
        return match ($request->user()->role) {
            'admin'    => redirect()->route('admin.panel'),
            'docente'  => redirect()->route('docente.panel'),
            default    => redirect()->route('estudiante.panel'),
        };
    }
}
```

7.3 Rutas con protección por rol

En `routes/web.php`, reemplaza la ruta `dashboard` de Breeze y añade los paneles. Conserva `require __DIR__.'/auth.php';` al final:

ROUTES/WEB.PHP

```

use App\Http\Controllers\DashboardController;

Route::get('/dashboard', [DashboardController::class, 'index'])
    ->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    Route::view('/admin', 'panels.admin')
        ->middleware('role:admin')->name('admin.panel');

    Route::view('/docente', 'panels.docente')
        ->middleware('role:admin,docente')->name('docente.panel');

    Route::view('/estudiante', 'panels.estudiante')
        ->middleware('role:admin,docente,estudiante')->name('estudiante.panel');
});

```

Cómo se leen los roles del middleware

En `role:admin,docente`, todo lo que sigue a los dos puntos llega al middleware como parámetros `...$roles`. El usuario pasa si su rol coincide con cualquiera de ellos.

7.4 Vistas de los paneles

Crea la carpeta `resources/views/panels/` con una vista por rol. Ejemplo para `panels/admin.blade.php` (replica el patrón para docente y estudiante):

```

RESOURCES/VIEWS/PANELS/ADMIN.BLADE.PHP
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-brand leading-tight">
            {{ __('Panel de Administración') }}
        </h2>
    </x-slot>

    <div class="py-12">
        <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white shadow sm:rounded-lg p-6">
                Bienvenido, {{ auth()->user()->name }}.
                Tu rol es <strong>{{ auth()->user()->role }}</strong>.
            </div>
        </div>
    </div>
</x-app-layout>

```

7.5 Navbar condicionado por rol

En `resources/views/layouts/navigation.blade.php`, dentro del bloque de enlaces de navegación, muestra cada enlace según el rol:

```

RESOURCES/VIEWS/LAYOUTS/NAVIGATION.BLADE.PHP
<div class="hidden space-x-8 sm:-my-px sm:ms-10 sm:flex">
    @if (auth()->user()->isAdmin())
        <x-nav-link :href="route('admin.panel')"
            :active="request()->routeIs('admin.panel')">
            {{ __('Administración') }}
        </x-nav-link>
    @endif

    @if (auth()->user()->hasRole('admin', 'docente'))

```

```
<x-nav-link :href="route('docente.panel')"  
  :active="request()->routeIs('docente.panel')">  
  {{ __('Docencia') }}  
</x-nav-link>  
@endif  
  
<x-nav-link :href="route('estudiante.panel')"  
  :active="request()->routeIs('estudiante.panel')">  
  {{ __('Mi Portal') }}  
</x-nav-link>  
</div>
```

Prueba el flujo completo

Inicia sesión con `admin@uatf.edu.bo` (rol admin) y con un usuario registrado como estudiante. Verifica que cada uno aterriza en su panel y que los enlaces del navbar cambian según el rol. Intenta acceder manualmente a `/admin` con un estudiante: debes recibir un 403.

► Paso 8 — Verificación final

Confirma que cada elemento de la guía funciona correctamente:

- ✓ **Breeze instalado y migraciones aplicadas:** Registro e inicio de sesión funcionan.
- ✓ **Branding aplicado:** Logo propio, botón y fondo con colores de marca.
- ✓ **Interfaz en español:** Etiquetas, botones y mensajes de validación traducidos.
- ✓ **Campo de rol:** El registro guarda el rol; no permite elegir admin.
- ✓ **Redirección por rol:** Cada rol aterriza en su panel tras el login.
- ✓ **Autorización:** Acceder a un panel sin permiso devuelve 403.

5. Rúbrica de evaluación

Criterio	Detalle	Pts.
Instalación de Breeze	Proyecto Laravel 13 con Breeze (Blade), migraciones y assets funcionando.	20
Branding	Logo propio + colores de marca aplicados a botón y fondo.	15
Localización	Interfaz y mensajes de validación en español.	15
Campo de rol	Migración, modelo, vista y controlador correctos.	15
Seguridad del rol	Validación con Rule::in y admin solo por seeder.	10
Redirección y middleware	Redirección por rol y protección de paneles (403).	15
Orden y entrega	Repositorio limpio, commits y tag correctos.	10

TOTAL

100

6. Entregables y entrega

Repositorio. Sube el proyecto a un repositorio nuevo llamado `portal-breeze`. Realiza commits descriptivos por cada personalización (branding, localización, rol, navbar).

Etiqueta (tag). Al concluir la guía, crea la etiqueta `v1.0.0`, que marca la primera versión funcional del portal con autenticación personalizada:

TERMINAL

```
git add .
git commit -m "Guía 8: Breeze + personalización (branding, i18n, rol, navbar)"
git tag -a v1.0.0 -m "Portal con autenticación Breeze personalizada"
git push origin main --tags
```

Formato de entrega

Entrega la URL del repositorio con la etiqueta `v1.0.0` visible. Incluye en el README las credenciales del administrador sembrado y una captura de cada panel por rol.

7. Referencias

Documentación de Laravel Breeze (Starter Kits). laravel.com/docs/11.x/starter-kits

Localización de Laravel (Localization). laravel.com/docs/13.x/localization

Middleware en Laravel. laravel.com/docs/13.x/middleware

Tailwind CSS — Tema y personalización de colores. tailwindcss.com/docs/theme

Laravel Lang — traducciones de la comunidad. github.com/Laravel-Lang/lang